IDEA Student Client Image – Design

One repeatable Ubuntu card for shared classroom Pis, reflashed remotely when it goes wrong

Proposal · Status: Approved by Koen 2026-10-15

IDEA - offline school computing. Prepared for Koen Swings's team.
No real passwords, keys or tokens are included; values in <ANGLE BRACKETS> are placeholders.

Contents

1. Problem 3
2. Summary of the proposal 3
3. Ubuntu flavour, version and hardware 4
4. Accounts and security 4
5. Reset: reflash (A) versus "clean on every reboot" (B) 5
6. Applications 6
7. Updates: off on the clients 6
8. Network, names and first boot 7
9. Build pipeline 7
10. Flashing from the Console 7
    10.1 The image as a versioned artifact on every Engine 7
    10.2 The Flash operation 8
    10.3 Safety without a reader allowlist (Koen's question 9) 8
11. Implementation plan 9
12. Automated tests (nothing manual for Koen) 9
13. Decisions (open questions closed, 15 Oct 2026) 10

Author: Steve (Lead Bot) **Date:** 2026-10-15 **Status:** Approved by Koen 2026-10-15 (revision 3: no flash size limit; large SSDs allowed)

1. Problem

Classroom computers (**Raspberry Pi 400 and Pi 500**, also Pi 4 / Pi 5, each with a 32 GB microSD card) are **shared**: a different student sits at each one every lesson. Today each card is set up by hand from Manual 2 (`docs/manuals/install/install-manual-client-pi400/`), including the Ubuntu first-boot wizard. That is slow, every card ends up slightly different, and when a card breaks (power cut, full disk, a student changed settings) a local helper has no simple way back to a known-good state.

Koen's requirements (15 October 2026):

- One **repeatable** Ubuntu image, the same on every student computer, **Pi 400 and Pi 500 alike**.
- **No personal accounts.** One `learner` account with no password.
- A **standard set of applications**: office (Word/Excel compatible), education software, and games.
- **Reset = reflash.** A card goes into a card reader on a school server Pi and Koen, or a local support engineer, writes a fresh default image **remotely over Tailscale**, from the **Console**.
- **No automatic updates** on the clients.
- **Every Engine can flash**: the standard image is stored on the Engine; no fixed list of approved readers.

2. Summary of the proposal

1. **Ubuntu Desktop 26.04 LTS for Raspberry Pi**, customised by a **build script** into `idea-client-<version>.img.xz` with a SHA-256 checksum. One image boots on Pi 4, 400, 5 and 500 (§3).
2. **One `learner` account, automatic login, no password**, no sudo. A separate `admin` account for support: SSH by key only, plus a local password known only to support (§4).
3. **Updates off** on the clients. A new tested image version, reflashed, is the only way software changes (§7).
4. **Flash from the Console.** The image is a **versioned, checksummed artifact stored on the Engine**, spread to every Engine like an app. The Console lists cards in any card reader and offers **Flash** with a clear confirmation; the Engine writes and verifies (§10).
5. **No reader allowlist.** Instead the Engine **stops claiming unknown disks automatically**: a disk without IDEA `META.yaml` is never mounted read-write or written to until someone initialises it in the Console. Flash targets are picked explicitly, with hard refusals for IDEA disks, the system disk and anything in use (§10.3).
6. Reset level B ("clean on every reboot", read-only root) is built in but **off in v1**; it can be switched on later remotely, per computer, without a reflash (§5).

3. Ubuntu flavour, version and hardware

Option	For	Against	Verdict
Ubuntu Desktop 26.04 LTS (GNOME), preinstalled raspi image	Official image, 5 years of support, same as Manual 2	GNOME uses ~1.2-1.5 GB RAM idle; heavier on a 4 GB Pi 400	**Decided**
Ubuntu MATE / Xubuntu 26.04 for Pi	Lighter	Community images, less tested	Fallback if the boot test shows GNOME is too slow
Ubuntu 24.04 LTS / Raspberry Pi OS	-	Older, or not Ubuntu	No

One card for Pi 400 and Pi 500 (Koen's question 8): yes. Canonical publishes a single `ubuntu-26.04-preinstalled-desktop-arm64+raspi.img.xz` "for modern Raspberry Pi boards (Pi 3, 4, 5, ...)", and the Ubuntu hardware support matrix lists 26.04 LTS as supported (server and desktop) on the **Pi 400 and the Pi 500** (Pi 4B and Pi 5 also certified). The image carries the boot firmware and kernel for both families and picks the right device tree at boot, so the same card moves freely between a Pi 400 and a Pi 500. Two conditions:

- **Bootloader EEPROM** must be recent enough: at least 2022-11-25 for Pi 4/400 and 2025-02-11 for Pi 5/500 (Ubuntu's documented minimum). New Pi 500s ship newer; older Pi 400s may need a one-time EEPROM update with the Raspberry Pi bootloader-update card. Manual 2 gets a "check the EEPROM date" step.
- **Keyboard layout** differs per kit (UK/US ANSI etc.), not per model; it stays a per-school build parameter.

Our own customisation adds no model-specific parts. The boot test (§12) includes an explicit **cross-model swap**: the same card is first-booted on a Pi 400 then moved to a Pi 500, and the reverse, to prove first boot does not make the card model-specific.

Small tweaks that help a 4 GB Pi: zram swap, GNOME search indexer (Tracker) off, animations off.

4. Accounts and security

Account	Purpose	Login	Rights
`learner`	Every student	**Automatic login, no password**	No sudo, cannot install software or change network/system settings
`admin`	Koen / local support engineer	**SSH by key only** (via the school server Pi); at the keyboard: local password known to support, kept in the support vault, never in docs or git	sudo

Passwordless `learner` is safe enough for a shared lab machine with no private data: what matters is that `learner` **cannot break the system** (not in `sudo`/`adm`/`lpadmin`; polkit denies network, software, user and power changes; screen lock off; Wi-Fi password stored root-only). SSH: `PasswordAuthentication no`, `AllowUsers admin`. Students save to **Nextcloud** on the school Pi; a local Documents folder is wiped by a reflash.

Can a support engineer open a terminal as `learner` and install software? (Koen's question 6) They can open a terminal (Ctrl+Alt+T), but **not install**: `learner` has no sudo, on purpose, otherwise any student could do the same. Decision:

- **At the computer:** in the learner's terminal type `su - admin`, enter the support password, then use `sudo`. No logout needed, and the student account stays locked down. The login screen does not show `admin` (hidden user), so students do not see it.
- **Remotely:** `ssh -J <school-pi> admin@<client>` over Tailscale.
- **But:** an install on one computer is **lost at the next reflash** and makes that card different from the rest. Use it for diagnosis or an urgent fix only. Software that should stay goes into `client-image.conf` and a new image version. Schools are offline, so `apt install` also needs a package copied in by hand; support docs say so.

5. Reset: reflash (A) versus "clean on every reboot" (B)

	A. Reflash the card	B. Read-only root + overlay (`overlayroot`)
How	Card into a reader on the server Pi, **Flash** in the Console, card back	Root mounted read-only; changes go to RAM and vanish at reboot
Fixes	Everything, including a corrupted card	Changed settings, junk, broken desktop: a reboot fixes it
Effort per incident	Someone handles the card (5-10 min)	Just reboot
Power-loss robustness / card wear	Normal	Much better: the card is almost never written
Downsides	Needs hands at the computer	Local Documents also vanish; RAM use grows during a session

Decided: both are in the image. A is always available. B is **off in v1**, tried for two weeks on a test Pi, then decided per school.

Starting with A and switching to B in the field, remotely (Koen's question 7). B is already installed in every image, only switched off by one line in `/etc/overlayroot.conf`. So switching needs **no card handling**:

1. **Remote, per computer (no reflash):** Koen connects over Tailscale to the school server Pi, then `ssh -J <school-pi> admin@<client>` and runs `sudo idea-client-mode overlay` (sets `overlayroot="tmpfs"`) and reboots. Back to A: `sudo overlayroot-chroot idea-client-mode normal` and reboot. A script on the server Pi can do this for all clients in one go while they are switched on.
2. **For new and reflashed cards:** the image build has `OVERLAY=on|off`; once B is the school default, the next image version ships with B on, and every card flashed from the Console gets it.
3. **Later, from the Console:** a "Client mode" toggle could run step 1 for the school; not in v1.

6. Applications

All packages are in Ubuntu main/universe for **arm64** and installed as **.deb**, not snaps (CI fails if one is missing).

Group	Apps	Notes
Office	LibreOffice Writer, Calc, Impress, fonts Carlito/Caladea	Default save format .docx/.xlsx/.pptx
Browser	**Firefox** (.deb, not snap)	Bookmarks: IDEA Console, Kolibri, Nextcloud, Kiwix on the school Pi
Education	GCompris, Tux Paint, TuxMath, TuxType, KTurtle, Kalzium, Marble, Stellarium	Stellarium/Marble slow-ish on Pi 400, fine on Pi 500
Maths / coding	**GeoGebra** and **Scratch 3** (no arm64 packages)	Served as **offline web apps** from the school Pi and bookmarked. GeoGebra licence: **confirmed OK** by Koen
Games	SuperTuxKart, GNOME Chess (+ stockfish), Mines, Sudoku, Aisleriot, Quadrapassel	SuperTuxKart at low settings
Language	English UI, `hunspell-en-gb`; Shona/ Ndebele where available	Keyboard per school build parameter

Removed: Ubuntu Pro prompts, Thunderbird, Software Center, update notifier, first-boot wizard, crash reporter popups.

7. Updates: off on the clients

Koen's requirement (question 3): updates are off, after a past update made a system unusable. **Our opinion: we agree.** For shared, offline classroom computers a silent update is a bigger risk than an old package. An update can half-finish at a power cut, fill a 32 GB card, or change the desktop overnight, and it makes cards differ from each other. The image therefore:

- removes `unattended-upgrades` and `update-notifier`, disables `apt-daily.timer` and `apt-daily-upgrade.timer`, and sets `APT::Periodic::*` to `0`;
- has no snaps in the student path; any snap left in the base is held with `snap refresh --hold` (forever) and `snapd` refresh timers off;
- has no Software Center and no firmware updater (`fwupd` off).

How software changes instead: we build a **new image version**, test it on the pool (boot test, §12), publish it to the Engines (§10.1), and schools reflash when convenient. Every computer is then on a known, tested version, and rolling back means flashing the previous version (Engines keep the last two).

Security trade-off: the clients miss security patches between image versions. We accept that: they are offline, on the school's IDEA network only, hold no private data, students have no admin

rights, and a reflash fixes any compromise. We plan a new image at least twice a year (and sooner for a serious browser issue).

8. Network, names and first boot

- Clients join the **IDEA Wi-Fi**; the profile is in the image, the Wi-Fi password is a per-school build parameter (not in git).
- The school Pi is at the fixed address **10.42.0.2**; bookmarks use that address.
- **Names:** every card is identical; first boot names the computer `idea-pc-<last 4 of the Pi serial>` (shown on a sticker).
- First boot regenerates SSH host keys and machine-id; the root partition grows to fill the card.
- Locale en_GB, time zone Africa/Harare (build parameters). Power loss: ext4 + `fsck` on boot; mode B removes most risk.
- **Tailscale on clients: no** (decided); support goes through the school server Pi as a jump host.

9. Build pipeline

1. `tools/client-image/build.sh` in the `idea` repo downloads the official raspi desktop image, checks Ubuntu's SHA256SUMS, loop-mounts it and chroots with `qemu-user-static` to install packages and apply the configuration.
2. All settings in one versioned file `client-image.conf`. Wi-Fi password and support keys/password hash are injected at build time from the ops box, never committed.
3. Output: `idea-client-26.04-<YYYYMMDD>-<git sha>.img.xz` + `.sha256` + `MANIFEST.txt` (all package versions).
4. CI builds on every change; a version is **published** only after the boot test passes (§12).

10. Flashing from the Console

10.1 The image as a versioned artifact on every Engine

- The client image is distributed **like an app**: a catalogue entry `client-image` with version, size and SHA-256, stored on the Engine under `/var/lib/idea/client-images/<version>/` (or on the Files Disk when the system disk is small).
- It reaches schools the way apps do: copied from a disk (e.g. the update stick or a Files Disk) or from another Engine on the network. The Engine **verifies the SHA-256 on arrival** and refuses a mismatched file.
- Every Engine keeps the **current and previous** version, so any school server Pi can flash, offline, and roll back.

10.2 The Flash operation

- The Console shows a **Client cards** section per Engine: every card in a card reader, with reader name, card size, and what is on it now (e.g. "Ubuntu client image 2026-10-01", "Raspberry Pi OS", "empty").
- **Flash** button: choose the image version (default: current) and confirm in a dialog that repeats the card size, reader and current contents, and says the card will be wiped.
- The Engine does the write through a fixed **root helper** (`flash-card`, same pattern as the existing helpers): re-checks the target, checks the image SHA-256, writes, **reads back and verifies**, flushes, powers the card off, and reports progress and "Done - safe to remove" to the Console. Each flash is logged (who, when, which image, which card).
- Remote use (Koen or support): open the Console over Tailscale, ask someone at school to put the card in, press Flash.

10.3 Safety without a reader allowlist (Koen's question 9)

Koen rejected a fixed list of approved readers as inflexible. Instead the safety comes from **two rules in the Engine**, which work with any reader in any Engine:

1. **The Engine no longer claims unknown disks by itself.** Today it mounts any new USB disk read-write and writes a `META.yaml` on it (seen on wizardly-hugle on 15 Oct: the card in the new reader was mounted at `/disks/sdc2` and got a `META.yaml`). New rule: a disk **without** IDEA `META.yaml` is only looked at (partitions and labels read, mounted read-only at most, as already done for FAT sticks) and shown in the Console as a new disk. It gets `META.yaml` only when someone chooses **Initialise** (or an app install) in the Console. This also protects any other foreign disk.
2. **Flash targets are chosen explicitly and checked hard.** A device is offered as a flash target when it is removable media (kernel `removable=1`, typically a USB card reader or an external USB SSD). There is **no size limit**: a large SSD is allowed when Koen or support chooses it on purpose. Whatever the Console shows, the helper **refuses** to write to: the system disk, any disk with an IDEA `META.yaml`, any disk with a mounted partition or a running app, and the device the Engine itself runs from. The confirmation dialog always shows the target **name and size**. The target is resolved again from `/dev/disk/by-id/` at write time, and the write is aborted if the medium was swapped.

Result: any reader works, plugged into any Engine, with no setup. A card is never written by accident, because the only write paths are an explicit Console Initialise or an explicit, confirmed Flash, and IDEA disks are never flash targets.

Engine change needed (Axle): rule 1 (no automatic `META.yaml` / read-write mount for unknown disks; Initialise in the Console), the client-card classification, the `flash-card` helper call and progress API, and the image artifact (store, verify, sync). Console (Pixel): Client cards section, Flash dialog, Initialise for new disks.

11. Implementation plan

Who	Work
Axle (Engine)	Rule 1 (no auto-claim of unknown disks), client-card detection, flash API + progress, client-image artifact store/verify/sync; unit tests + hardware test on a pool Pi
Atlas (ops)	`flash-card` root helper, image storage paths, deployment to pool and wizardly-hugle; until the Console flow exists, flashes by hand with the same helper
Kid (apps / content)	Build script and `client-image.conf`, updates-off settings, offline Scratch/GeoGebra bundles
Pixel (Console)	Client cards section, Flash dialog with confirmation and progress, Initialise for new disks
Marco (field docs)	Manual 2 update (see follow-up below) and a one-page "reset a student computer" sheet
Steve	Coordination, Design Review, issues

Follow-up: Manual 2 name. The client install manual lives in `docs/manuals/install/install-manual-client-pi400/` (`install-manual-client-pi400.md`, its `build.py`, PDF, diagrams and the generator script), and is referenced from `docs/INDEX.md`, `docs/DOCUMENTATION-RULES.md` and `docs/manuals/install/README.md`. It must not be Pi 400-specific. Proposal: rename to **`install-manual-client`** ("Manual 2 - Student client computers (Pi 400 / Pi 500)"), update all references, and add the Pi 500 and EEPROM steps. Not done in this PR (seven files plus PDFs and diagrams); it goes with Marco's manual update once the image exists.

12. Automated tests (nothing manual for Koen)

- **Build CI:** fail if a package is missing for arm64, `learner` has sudo, SSH allows passwords, any update timer or `unattended-upgrades` is active, a snap is not held, or a secret pattern is found.
- **Boot test** on a pool Pi with a reader: flash through the Engine helper, boot on a **Pi 400 and a Pi 500** (USB boot from the reader); check autologin, Firefox opens the Console, LibreOffice saves a .docx, `learner` cannot `sudo`, `su - admin` works, no apt activity after 24 h, mode B discards a test file after reboot.
- **Cross-model swap (Koen, 15 Oct):** prove the card does **not** become model-specific after first boot (first-boot resize, cloud-init, firmware/`config.txt`, initramfs, or device-tree choices). Two directions, both required: 1. Flash a fresh card → **first boot on a Pi 400** → use it (learner autologin, open apps, reboot once) → move the same card to a **Pi 500** → full boot and function. 2. Flash another fresh card → **first boot on a Pi 500** → same use and reboot → move to a **Pi 400** → full boot and function. **Pass:** in both directions the second model reaches the learner desktop, network (IDEA Wi-Fi / school Pi bookmarks) works, and the standard apps start. **Fail:** any stuck boot, missing desktop, broken network, or apps that only work on the first-boot model.
- **Engine tests:** an unknown disk is not written to and gets no `META.yaml`; flash refuses an IDEA disk, the system disk, a mounted disk and a corrupted image; a swapped medium aborts the write. A large SSD is accepted when chosen explicitly (no size limit).

13. Decisions (open questions closed, 15 Oct 2026)

Koen agreed with all recommendations. Recorded as decided:

1. **Passwordless autologin for `learner`:** yes.
2. **Overlay mode (B) by default:** off in v1, two-week trial, then decide; switchable remotely (§5).
3. **Desktop:** GNOME; the boot test's memory numbers can still move us to a lighter desktop.
4. **Tailscale on every client:** no; the school server Pi is the jump host.
5. **Where things live:** build in the `idea` repo under `tools/client-image/`; images on the Engines (§10.1), not in git.
6. **GeoGebra offline licence:** OK (Koen). Scratch is open source.
7. **Which server Pi flashes:** any Engine (no dedicated Pi); on wizardly-hugle only after the Engine rule-1 change, since it runs Milkwise.
8. **The old Raspberry Pi OS card on wizardly-hugle:** may be overwritten as the first test card.

Further decisions from Koen's feedback: Pi 500 is a client; one image for Pi 400 and Pi 500; updates off; flash from the Console with the image stored on every Engine; account `admin`; no reader allowlist; cross-model swap test required (Pi 400→500 and 500→400 after first boot).

9. **No flash size limit (Koen, 2026-10-15 09:41):** do **not** refuse targets above 256 GB. Koen sometimes wants to flash a large SSD. Remaining guards only: never a disk with IDEA `META.yaml`, never the system disk, never a disk in use or mounted, and an explicit Console confirmation that shows the target name and size.